

Esame Programmazione di Sistema / API Programming

Data: 2025-07-03



PORTALE
ESAMI



PORTALE
ESAMI



QUESTIONS

Domanda 1 (Teoria)

1. Si descriva il meccanismo di gestione degli errori fornito dal linguaggio RUST, differenziando tra quelli recuperabili e irrecuperabili.
2. Si spieghi il funzionamento dell'operatore ?.
3. Si mostri un esempio di applicazione dell'operatore ? scrivendo una funzione (avendo cura di scriverla completamente) che deve cercare una stringa (`my_string`) e restituire il suo offset in un file, se presente.
Per leggere il file si deve usare la funzione:
`fn read_file(path: &str) -> Result<String, std::io::Error>`
che legge il file di nome `path` e restituisce l'intero contenuto del file in una `String`, se non ci sono errori.
Per la ricerca della stringa si faccia uso del seguente metodo:
`fn find(&self, pat: P) -> Option<usize> where P: Pattern<'a>`
che restituisce l'offset della stringa, se trovata.
4. Si descrivano i principali vantaggi ed eventuali svantaggi rispetto ad altri linguaggi di programmazione in termini di organizzazione del codice e prestazioni.

Domanda 2

A. Completa questa funzione in modo che compili correttamente, se ritieni che ci sia un errore di compilazione (descrivendolo), altrimenti dichiara il motivo per cui la funzione è scritta correttamente:

```
1 fn get_prefix(s: &str, len: usize) -> &str {  
2     &s[..len]  
3 }
```

B. Data la struct seguente:

```
1 struct Container<'a> {  
2     data: &'a str,  
3 }  
4 impl<'a> Container<'a> {  
5     fn get(&self) -> &str {  
6         self.data  
7     }  
8 }
```

Il metodo `get()` necessita di specificare esplicitamente il tempo di vita del valore ritornato? Motivare la risposta affermativa o negativa.

C. Il codice seguente compila? Se la risposta è negativa, correggilo e spiega il motivo dell'errore di compilazione.

```
1 fn make_closure(s: &str) -> impl Fn() -> &str {  
2     move || s  
3 }
```

D. Considerare la seguente funzione:

```
1 fn concat_parts<'a>(a: &'a str, b: &'a str) -> &'a str {  
2     let c = String::from(a) + b;  
3     &c  
4 }
```

Il codice è correttamente compilato? Si giustifichi la risposta. Nel caso negativo, si fornisca una soluzione corretta, evitando di introdurre copie inutili delle stringhe, ma eventualmente modificando la signature della funzione.

Domanda 3

A. Si consideri il seguente frammento di programma Rust:

```
1 fn main() {  
2     let mut s = String::from("esame");  
3     let r1 = &s;  
4     let r2 = &mut s;  
5     r2.push_str(" superato");  
6     println!("r1: {}", r1);  
7     println!("s: {}", s);  
8 }
```

1. Analizza il codice riga per riga. Quale errore segnalerà il compilatore Rust e perché? Indica la riga precisa in cui l'errore viene rilevato.
2. Riscrivi il codice in modo che sia corretto e produca il seguente output, esclusivamente cambiando l'ordine delle istruzioni:

```
r1: esame  
s: esame superato
```

B. Considera il seguente frammento di codice Rust:

```
1 fn process_data(data: &mut String) {  
2     data.push_str(" processed");  
3 }  
4 fn analyze_data(data: &String) -> usize {  
5     data.len()  
6 }  
7 fn main() {  
8     let mut s = String::from("original data");  
9     let r1 = &s;  
10    println!("Length: {}", analyze_data(r1));  
11    let r2 = &s;  
12    println!("Content (r2): {}", r2);  
13    process_data(&mut s);  
14    println!("Final string: {}", s);  
15    println!("Length from r1 (after processing): {}", analyze_data(  
16        r1));  
17 }
```

1. Quale errore specifico di compilazione (o errori) verrà generato dal borrow checker in questo codice? Qual è la riga o le righe incriminate?
2. Come potresti modificare il codice in modo minimale per farlo compilare correttamente? Si modifichi il codice mantenendo l'intento di processare e poi analizzare i dati con il seguente output:

```
Length: 13  
Content (r2): original data  
Final string: original data processed  
Length from s (after processing): 23
```

Programming Question: Aggregatore di misure

Un sistema di monitoraggio all'interno di uno stabilimento industriale raccoglie misure di temperatura da più sensori. Le misure vengono raccolte in modo asincrono, sono automaticamente etichettate con l'istante temporale in cui sono comunicate e possono essere inviate da più thread contemporaneamente. Compito del sistema è quello di aggregare le misure ricevute, calcolando la temperatura media e il numero di misurazioni ricevute da ciascun sensore, operando un campionamento ad intervalli regolari indicati dal parametro passato alla funzione di costruzione. In tale periodo, un sensore può inviare più misure, che devono essere tutte considerate nel calcolo della media. Un thread interno alla struttura si occupa di calcolare la media delle temperature per ciascun sensore, aggiornandola secondo il periodo di campionamento indicato. All'atto della distruzione della struttura, il thread interno deve essere terminato in modo sicuro.

Per implementare tale sistema, si richiede di realizzare la struct **Aggregator** che offre i seguenti metodi thread-safe:

```
1 use std::time::Instant;
```

```

2
3 pub struct Aggregator {
4     // campi privati
5 }
6
7 pub struct Average {
8     pub sensor_id: usize,
9     pub reference_time: Instant, // istante in cui e' stata
    calcolata la media
10    pub average_temperature: f64,
11 }
12
13 impl Aggregator {
14     pub fn new(sample_time_millis: u64) -> Self {
15         // implementazione del costruttore
16         todo!()
17     }
18
19     pub fn add_measure(&self, sensor_id: usize, temperature: f64) {
20         // aggiunge una misura di temperatura per il sensore con id
        'sensor_id'
21         // e temperatura 'temperature'. Le misure sono
        automaticamente etichettate
22         // con l'istante temporale in cui sono comunicate.
23         todo!()
24     }
25
26     pub fn get_averages(&self) -> Vec<Average> {
27         // restituisce un vettore che riporta la temperatura media
        di ciascun sensore,
28         // calcolata durante l'ultimo periodo di campionamento.
29         // Sono presenti solo i sensori che hanno inviato almeno
        una misura.
30         todo!()
31     }
32 }

```

SOLUTIONS

Domanda 1 (Teoria)

1. In Rust, la gestione degli errori distingue tra errori irrecuperabili e recuperabili. Gli errori **irrecuperabili** si verificano in situazioni anomale critiche (es. array out of bounds) e arrestano immediatamente il programma (o il thread corrente) tramite la macro `panic!()`. Gli errori **recuperabili** sono rappresentati dall'enum `Result<T, E>`. Questo approccio obbliga il programmatore a gestire esplicitamente la possibilità di errore: `Ok(T)` contiene il valore di successo, mentre `Err(E)` contiene i dettagli del fallimento, permettendo al chiamante di decidere come agire di conseguenza senza interrompere l'intero processo.
2. L'operatore `?` è uno zucchero sintattico (syntactic sugar) per la gestione rapida e leggibile degli errori recuperabili propagabili. Può essere apposto dopo un'espressione che restituisce un `Result`. Se l'espressione valuta `Ok(val)`, `?` restituisce `val` all'istruzione corrente, permettendo all'esecuzione di continuare regolarmente. Se valuta `Err(e)`, `?` interrompe anticipatamente l'esecuzione della funzione corrente e ritorna immediatamente `Err(e)` al chiamante. Se necessario, l'operatore `?` converte automaticamente l'errore `e` nel tipo di errore restituito dalla funzione (sfruttando il trait `From`).
3. Esempio di applicazione:

```
1 fn find_string(path: &str, my_string: &str) -> Result<Option<
  usize>, std::io::Error> {
2     // Uso l'operatore ? per propagare eventuali errori di IO
3     let content = read_file(path)?;
4     // Ritorno il risultato della ricerca avvolto in un Ok()
5     Ok(content.find(my_string))
6 }
```

4. **Vantaggi:** Chiarezza assoluta dell'API: dalla signature di una funzione è immediatamente evidente se essa può fallire. Gli errori non possono essere ignorati accidentalmente e la mancanza di costrutti nascosti di controllo del flusso rende il codice predicibile. Prestazionalmente parlando (a run-time), i controlli sono "zero-cost" rispetto a meccanismi pesanti basati sullo stack unwinding usati dalle eccezioni classiche (es. in Java o C++), e non richiedono frame table aggiuntive. **Svantaggi:** Maggiore verbosità, necessita di gestire esplicitamente i tipi di errore. Se in una funzione vengono richiamate più funzioni che ritornano errori di tipo diverso, può essere difficile comporli in un unico tipo restituito (e richiede boilerplate code o l'uso di librerie esterne come `anyhow` o `thiserror`).

Domanda 2

A. La funzione sintatticamente compila in maniera corretta grazie alle regole di *lifetime elision* (il parametro `s` acquisisce un lifetime anonimo che viene poi implicitamente applicato al valore di ritorno). Tuttavia, a livello semantico e logico, `&s[..len]` affetta la stringa contando i *bytes* e non i *caratteri* UTF-8. Se `len` cade all'interno di un carattere multibyte, la funzione causerà un *panic* a runtime.

B. No, non e' necessario. Secondo le regole di *lifetime elision* di Rust, se esiste esattamente un input reference (`&self` in questo caso), il suo lifetime e' automaticamente assegnato a tutti gli output reference. Quindi il lifetime eliso nel tipo di ritorno `&str` sara' esattamente `'a`, lo stesso della variabile istanziata per `Container<'a>`.

C. No, non compila. Il compilatore rilevera' che la funzione ritorna un `impl Fn()` -> `&str` e che la stringa `s` viene catturata (con `move`) nella closure. Tuttavia, non riesce ad inferire il lifetime associato al valore di ritorno della chiusura (il lifetime di `s` dopo che viene spostato dentro l'oggetto closure). E' necessario specificare esplicitamente un lifetime per dire al compilatore che il riferimento restituito dalla chiusura vivra' tanto quanto l'argomento `s` fornito. Correzione:

```
1 fn make_closure<'a>(s: &'a str) -> impl Fn() -> &'a str {
2     move || s
3 }
```

D. No, non compila. Viene istanziata una nuova stringa di tipo `String` all'interno della funzione (variabile `c`), la quale alloca i suoi dati sull'heap. Successivamente, la funzione tenta di restituire un riferimento (`&c`) a tale stringa locale. Al termine della funzione, `c` esce dallo scope e viene deallocata, lasciando il riferimento pendente (dangling pointer). Il borrow checker blocca giustamente la compilazione.

Soluzione corretta: la soluzione e' modificare la signature della funzione affinche' restituisca il possesso dei dati (il tipo `String` e non una slice `&str`), evitando il dangling pointer.

```
1 fn concat_parts(a: &str, b: &str) -> String {
2     String::from(a) + b
3 }
```

Domanda 3

A.1 Il compilatore segnalera' un errore (borrow checker error) alla **riga 6**. Questo succede poiche' alla riga 3 creiamo un riferimento immutabile `r1` e alla riga 6 lo utilizziamo. Tra la sua creazione e il suo utilizzo (alla riga 4), creiamo pero' un riferimento mutabile `r2` e lo modifichiamo. Secondo le regole di Rust, **non e' possibile avere contemporaneamente un reference mutabile e uno o piu' reference immutabili attivi**. In particolare, la richiesta di creare il reference mutabile alla riga 4 entra in conflitto con l'uso successivo del reference immutabile.

A.2 Riscriviamo il codice per terminare anticipatamente il borrow immutabile `r1` (grazie alle Non-Lexical Lifetimes):

```
1 fn main() {
2     let mut s = String::from("esame");
3     let r1 = &s;
4     println!("r1: {}", r1); // Il borrow immutabile r1 termina qui
5
6     let r2 = &mut s;
7     r2.push_str(" superato"); // Il borrow mutabile r2 viene creato
8     // e usato
9     println!("s: {}", s);
10 }
```

B.1 Si generano due tipologie di errore alla riga 13 (`process_data(&mut s)`) e alla riga 15 (`println!("{}", analyze_data(r1))`). Alla riga 15 stiamo tentando di usare il riferimento immutabile `r1` (creato alla riga 9). Poiche' lo usiamo alla riga 15, il suo lifetime e' esteso perlomeno fino a quel punto. Ma alla riga 13 stiamo passando `&mut s` per processare i dati. Questo e' un conflitto perche' richiediamo un "mutable borrow" mentre un "immutable borrow" e' ancora attivo. Inoltre, se `println!` riceve `s` (non per reference) alla riga 14 (come c'era nella traccia, `println!("{}", s)`), avviene un passaggio di reference implicito ma si scontra col fatto che `r1` deve essere valido.

B.2 Correzione minimale: rinunciamo all'uso di `r1` alla riga 15, e passiamo direttamente un nuovo riferimento di `s` (`&s`) nella funzione `analyze_data`, in modo da consentire al reference immutabile `r1` di morire prima del borrow mutabile alla riga 13.

```

1 fn process_data(data: &mut String) {
2     data.push_str(" processed");
3 }
4 fn analyze_data(data: &String) -> usize {
5     data.len()
6 }
7 fn main() {
8     let mut s = String::from("original data");
9     let r1 = &s;
10    println!("Length: {}", analyze_data(r1));
11    let r2 = &s;
12    println!("Content (r2): {}", r2);
13
14    // Da qui r1 e r2 non sono piu' usati, il loro lifetime termina.
15    process_data(&mut s);
16
17    println!("Final string: {}", s);
18    // Utilizziamo un nuovo borrow immutabile su s per permettere la
19    // compilazione
20    println!("Length from s (after processing): {}", analyze_data(&s));
21 }

```

Programming Question: Aggregatore di misure

Per rispettare le direttive del testo ed evitare misurazioni vecchie infinite, la classe disporra' di un thread "background" che ciclicamente si sospende, elabora i campioni ricevuti all'interno della finestra temporale (`sample_time_millis`) e ne espone le medie. Utilizziamo `Arc<Mutex, Condvar>` in modo da implementare l'attesa interruptibile (permettendo la terminazione sicura tramite `Drop`).

```

1 use std::collections::HashMap;
2 use std::sync::{Arc, Condvar, Mutex};
3 use std::thread::JoinHandle;
4 use std::time::{Duration, Instant};
5
6 struct Measurement {
7     id: usize,
8     timestamp: Instant,
9 }

```

```

9     measure: f64,
10 }
11
12 struct InnerState {
13     running: bool,
14     measurements: Vec<Measurement>,
15     sample_time: Instant,
16     recent_averages: Vec<Average>,
17 }
18
19 #[derive(PartialEq, Clone, Debug)]
20 pub struct Average {
21     pub sensor_id: usize,
22     pub reference_time: Instant,
23     pub average_temperature: f64,
24 }
25
26 pub struct Aggregator {
27     state: Arc<(Mutex<InnerState>, Condvar)>,
28     join_handle: Option<JoinHandle<()>>,
29 }
30
31 impl Aggregator {
32     pub fn new(sample_time_millis: u64) -> Self {
33         let inner_state = Arc::new((
34             Mutex::new(InnerState {
35                 running: true,
36                 measurements: Vec::new(),
37                 sample_time: Instant::now(),
38                 recent_averages: Vec::new(),
39             }),
40             Condvar::new(),
41         ));
42
43         let state = inner_state.clone();
44         let join_handle = std::thread::spawn(move || {
45             let mut internal_state = state.0.lock().unwrap();
46             loop {
47                 let next_wakeup = internal_state.sample_time +
48                     Duration::from_millis(sample_time_millis);
49                 let sleep_time = next_wakeup.
50                     saturating_duration_since(Instant::now());
51
52                 // Attesa tramite condvar per svegliarsi al
53                 // termine del timeout
54                 // o se notificato prima per terminazione.
55                 internal_state = state.1.wait_timeout_while(
56                     internal_state, sleep_time, |s| s.running).
57                     unwrap().0;
58
59                 if !internal_state.running {

```

```

55         break;
56     }
57
58     internal_state.sample_time = next_wakeup;
59
60     // Estrazione delle misure vecchie dalla coda
61     // tramite approccio "drain in window".
62     // Evita di mantenere uno storico infinito.
63     let mut current_measurements = Vec::new();
64     let mut remaining_measurements = Vec::new();
65
66     for m in internal_state.measurements.drain(..) {
67         if m.timestamp < next_wakeup {
68             current_measurements.push(m);
69         } else {
70             remaining_measurements.push(m);
71         }
72     }
73     internal_state.measurements =
74         remaining_measurements;
75
76     // Rilascia il lock durante i calcoli onerosi per
77     // non bloccare l'aggiunta di misure
78     drop(internal_state);
79
80     let mut averages = HashMap::<usize, (f64, usize)>::new();
81     for m in current_measurements {
82         let entry = averages.entry(m.id).or_insert(
83             ((0.0, 0)));
84         entry.0 += m.measure;
85         entry.1 += 1;
86     }
87
88     let new_averages: Vec<Average> = averages.iter()
89         .map(|(id, (sum, count))| Average {
90             sensor_id: *id,
91             reference_time: next_wakeup,
92             average_temperature: sum / (*count as f64)
93         },
94         )
95         .collect();
96
97     // Riprende il lock per aggiornare le medie
98     // recenti da esporre
99     internal_state = state.0.lock().unwrap();
100     internal_state.recent_averages = new_averages;
101 }
102 });
103
104 Self {

```

```

99         state: inner_state,
100         join_handle: Some(join_handle),
101     }
102 }
103
104 pub fn add_measure(&self, sensor_id: usize, temperature: f64)
105 {
106     let now = Instant::now();
107     let mut state = self.state.0.lock().unwrap();
108     state.measurements.push(Measurement {
109         id: sensor_id,
110         timestamp: now,
111         measure: temperature,
112     });
113 }
114
115 pub fn get_averages(&self) -> Vec<Average> {
116     let state = self.state.0.lock().unwrap();
117     state.recent_averages.clone()
118 }
119
120 impl Drop for Aggregator {
121     fn drop(&mut self) {
122         let mut state = self.state.0.lock().unwrap();
123         state.running = false;
124         drop(state);
125         // Risveglia subito il thread per farlo uscire e
126         // concludere
127         self.state.1.notify_all();
128         if let Some(join_handle) = self.join_handle.take() {
129             let _ = join_handle.join();
130         }
131     }
132 }

```